

AgentVault — Whitepaper

On-chain policy vault for autonomous agents — give your agent an allowance, not a blank check.

Version 1 · Testnet 968 · September 2026

Abstract

AgentVault is a policy vault that lets autonomous agents spend on your behalf without giving them custody, a secondary key, or a blank check. The contract holds the funds. The owner writes a policy — which agents may act, which destinations they may pay, how much per token per day, how much per transaction, and how fast they may go — and the contract enforces that policy on every execution, with no popup, no second signature, and no middleman.

The owner controls the vault with an instant pause that carries zero delay, and a two-step, 48-hour timelocked ownership rotation. Agents execute inside the lines. They cannot step outside them, even if they try.

The problem: the price of agent autonomy is blank checks

As LLM agents start doing real things — moving money, paying vendors, filling orders — every integration face the same dilemma:

- **Give the agent a key.** It can do anything, any balance, any destination, any time. One confused prompt, one hallucinated address, one compromised context, and the whole wallet is gone.
- **Multi-sig every action.** You are the approval step, so the agent is not actually autonomous.
- **Trust a dashboard.** The "limits" live in UI code an agent (or a hijacked agent) can ignore.

The market needs something a script can honestly walk around *only within*: custody held by a contract, policy enforced by the same code, and limits that a single transaction cannot exceed.

The vault model

Custody

The vault holds the funds — native BOT and ERC-20. Agents never hold custody. They submit a request; the contract checks the policy; if the request passes, the contract transfers. If it does not, it reverts with the specific guardrail that fired.

Policy — written by the owner, enforced on-chain

Control	Function	Effect
Agents	<code>setAgent(address, bool)</code>	Allow or block an address from spending.
Destinations	<code>setTarget(address, bool)</code>	Allowlist where vault funds may be paid out.
Daily limit	<code>setDailyLimit(agent, token, limit)</code>	Per agent, per token, resets every 24h.
Per-tx cap	<code>setPerTxLimit(agent, token, cap)</code>	Ceiling on any single transfer; 0 = off.
Cooldown	<code>setExecCooldown(seconds)</code>	Minimum pause between executions.
Pause	<code>setPaused(bool)</code>	Freeze all spending instantly; state and limits untouched.

Every gate fires a named revert the agent can read and log — `TargetDenied`, `OverLimit`, `OverPerTx`, `CooldownActive` — so a blocked agent knows *why*, not just that it was stopped.

The four guardrails

1. **Allowlisted targets.** An agent can only pay addresses you approved. Anything else never leaves the vault.
2. **Daily limit.** A per-token ceiling that resets every 24 hours. Spend it down, requests wait for the next window.
3. **Per-transaction cap.** One runaway request cannot burn a day's allowance at once.
4. **Execution cooldown.** A minimum pause, so an agent firing faster than intended gets throttled, not emptied.

Ownership & safety

- **Instant pause, zero delay.** The moment something looks wrong, one call freezes spending. Pausing touches neither ownership nor limits, and unpausing restores them exactly.
- **Two-step, 48h rotation.** Ownership changes are proposed, wait a fixed period, and are completed only by the proposed owner. The current owner can still cancel outright during the wait. No single click hands the vault away.
- **Reentrancy-guarded.** Transfers are locked around settlement.

Architecture

Layer	Piece
Vault ledger	Balances: native + per-ERC-20; no address ever holds custody but the contract.
Policy engine	Agents, targets, daily limits, per-tx caps, cooldown — all owner-writable, all contract-read.
Execution loop	Request → four guardrail checks → transfer → state write, in one transaction.
Owner console	Dashboard reads every policy value live and returns the exact guardrail that fired.

Execution flow

```
owner: fund the vault — native BOT and/or ERC-20
owner: setAgent ✓ setTarget ✓ setDailyLimit ✓
      setPerTxLimit ✓ setExecCooldown ✓
agent: request(target, amount)
      ↳ target allowlisted? X TargetDenied
      ↳ under daily limit? X OverLimit
      ↳ under per-tx cap? X OverPerTx
      ↳ cooldown elapsed? X CooldownActive
      ↳ transfer executes ✓ one tx, no extra signature
owner: pause anytime — instant, revocable
owner: propose / cancel / accept owner — 48h timelock, only proposed owner
```

Use cases

- **Trading bots** — a routine that rebalances but cannot liquidate the vault in a single panic trade.
- **Payments assistants** — an agent that pays pre-approved vendors within a daily ceiling and can't mass-sweep.
- **Scheduled scripts** — cron-style jobs that pay out on cadence, throttled by cooldown, capped per run.

Roadmap

Item	Status
Policy vault: agents, targets, limits, cap, cooldown	Shipped (testnet 968)
Instant pause + two-step 48h rotation	Shipped
Native + ERC-20 deposits and readbacks	Shipped
Per-vault role/webhook receipts	Next
Allowlist bundles (pre-approved vendor sets)	Parked

Item	Status
Multi-vault manager for fleet operations	Parked

No token

AgentVault is infra, not a coin. No allocation, no fee, no governance token required to run a vault. Incentives sit with the ecosystem's points program, keeping the tool neutral for any agent framework or chain.

Disclaimer

This document describes a policy vault on BOT Chain testnet (968) and a live read-only frontend. Any deployed contract must be independently verified before sending real funds. Pause is instant; rotation is slow by design; nothing here is financial advice or a guarantee against market loss.